

EECS 482

Introduction to operating systems

Creating threads;
lock implementation

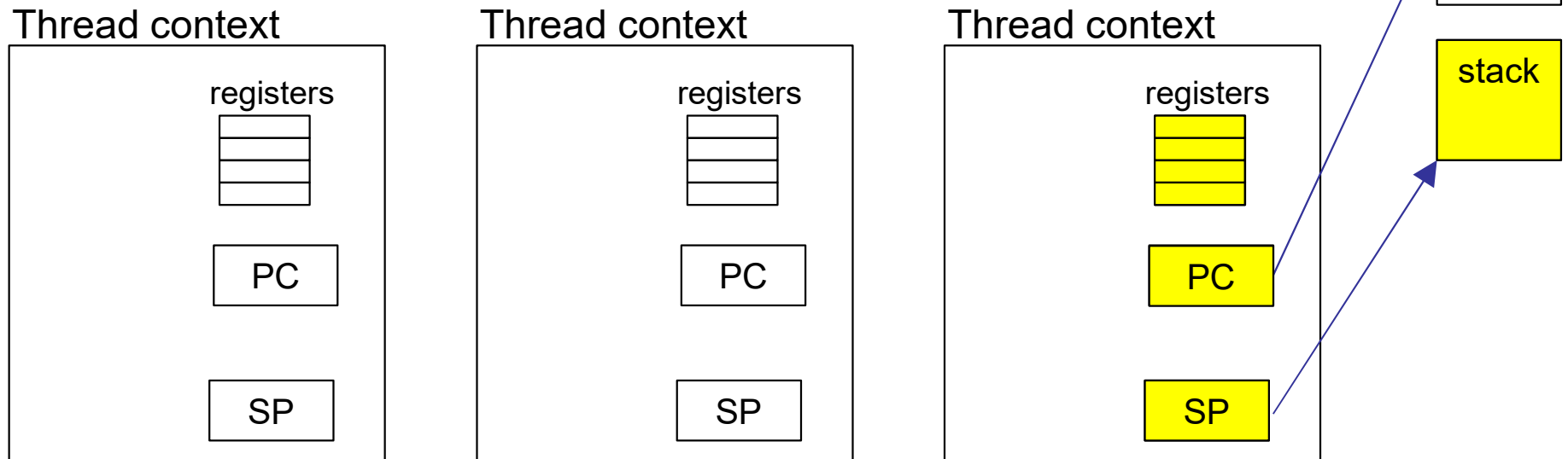
Peter Chen
University of Michigan

Creating a new thread

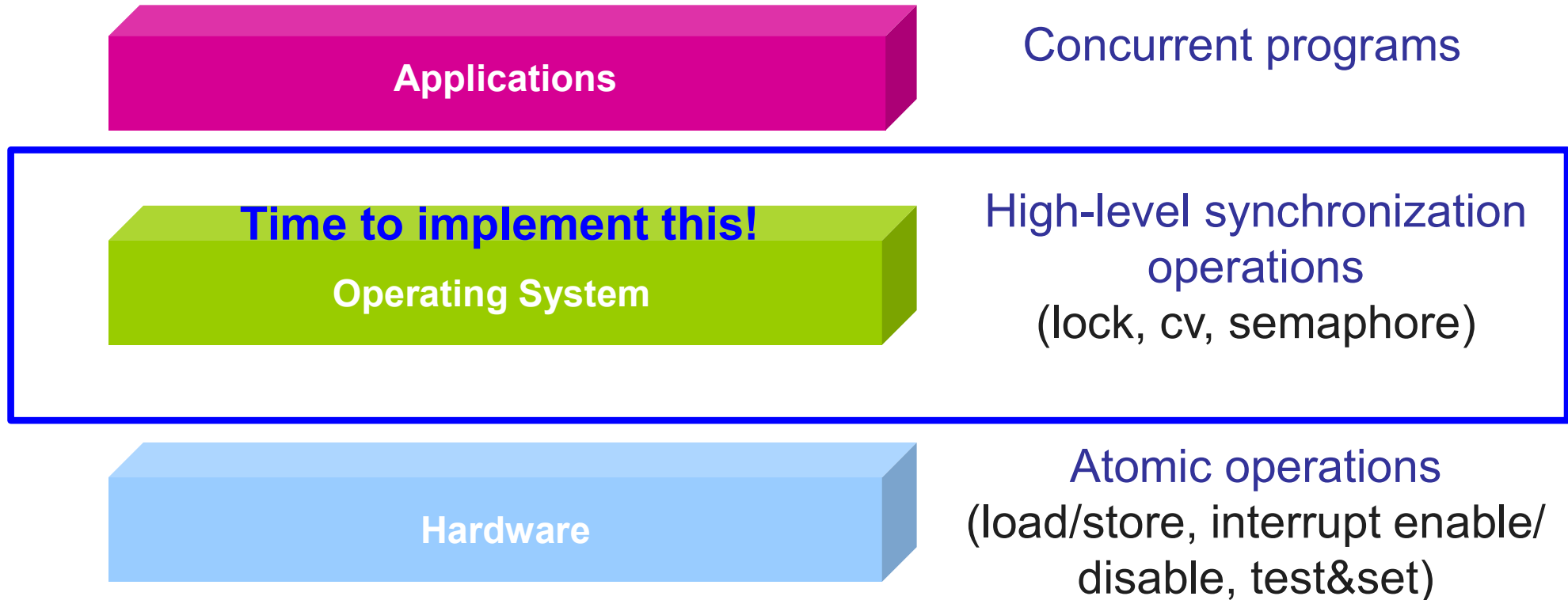
- Challenging to create a running thread
- Instead, create **ready** thread
 - Key idea: make it look like it was running, put on ready queue
 - Then just wait for it to be scheduled!
- Construct TCB in the state it would be in if it paused at start of its initial function

How to create a thread

Linux: `makecontext()` initializes a context's registers, PC, stack to the values they would have at the start of a function call



High-level synchronization



Implementing high-level synchronization operations (e.g., lock)

- Uses shared data
 - ➔ The code that implements these operations must be **thread safe**
- Use synchronization (mutual exclusion, ordering) to implement synchronization ?!
 - Can't use the normal high-level synchronization operations
 - Instead, use atomic operations provided by hardware, e.g., atomic load, atomic store, etc.

Writing OS code

- Disadvantage
 - OS code can't use high-level synchronization operations, since it is implementing these
- Advantages
 - OS trusts its own code (OS doesn't trust user code)
 - OS controls the hardware (e.g., CPU)

Remember our description of lock/unlock?

lock()

```
while (1) {  
    if (lock is free) {  
        acquire lock  
        break  
    }  
}
```

atomic

unlock()

```
release lock
```

atomic

Why is this not satisfactory?

Lock implementation #0

```
lock() {  
    while (status != FREE) {  
    }  
    status = BUSY  
}
```

```
unlock() {  
    status = FREE  
}
```

Problems?

How to provide atomicity for OS code

- Remember: can't use mutex or semaphores
- What breaks atomicity for a section of code?
 - ~~Code may call yield, etc.~~ Don't do this ☺
 - ~~An interrupt may occur~~ Disable interrupts around critical sections
 - ~~Another processor may execute instructions~~ Handled later
- How to fix each of these?

Atomicity on uniprocessor

- Prevent events that allow other threads to run
 - Don't call yield() in a critical section
 - Disable interrupts around critical section

- Example

disable interrupts

```
if (no milk) {  
    buy milk  
}
```

enable interrupts

- Problems?
 - Unsafe to run user code with interrupts disabled
 - How to provide multiple locks?

Who may disable interrupts?

- User code may not run with interrupts disabled
 - OS doesn't trust user code (to re-enable interrupts)
- But OS code may!
- Disable interrupts to provide atomicity for critical sections of OS code
- Disabling and enabling interrupts serves as a low-level lock for the OS

Lock implementation #1 (uniprocessor, busy waiting)

```
lock() {  
    disable interrupts  
    while (status != FREE) {  
        enable interrupts  
        disable interrupts  
    }  
    status = BUSY  
    enable interrupts  
}
```

```
unlock() {  
    disable interrupts  
    status = FREE  
    enable interrupts  
}
```

Busy waiting

- Problem with lock implementation #1
 - Waiting thread uses lots of CPU time just checking for lock to become free
 - Better for waiting thread to sleep and let other threads run
- Solution: integrate lock implementation with thread switching
 - `lock()` gives up CPU, so other threads can run
 - `unlock()` wakes up waiting thread when lock is free

Avoid busy waiting by switching threads

```
lock() {
    disable interrupts
    while (status != FREE) {
        enable interrupts
        add thread to queue of
    } threads waiting for lock
    status = BUSY
    switch to next ready thread
    enable interrupts
}
```

```
unlock() {
    disable interrupts
    status = FREE
    if (any thread is waiting
for this lock) {
        move waiting thread to
        ready queue
    }
```

- What does it mean for lock() to “add thread to queue of threads waiting for lock”?
- Why have a separate waiting queue for the lock? Why not put waiting thread onto ready queue?

When to re-enable interrupts?

```
lock() {  
    disable interrupts  
    while (status != FREE) {  
        ➡ enable interrupts  
        ➡ add thread to queue of  
           threads waiting for lock  
  
        switch to next ready thread  
        disable interrupts  
    }  
    status = BUSY  
    enable interrupts  
}
```

lock waiting queue

CPU

thread U

```
unlock() {  
    disable interrupts  
    status = FREE  
    if (any thread is waiting  
        for this lock) {  
        move waiting thread to  
        ready queue  
    }  
    enable interrupts  
}
```

ready queue

When to re-enable interrupts?

- Problem: adding thread to waiting queue and going to sleep must be **atomic**
- When have we seen this problem before?
- How did we solve it before?

Interrupt enable/disable pattern

- Adding thread to lock wait queue + going to sleep must be **atomic**
- Thread must **leave interrupts disabled** when calling switch

Leave interrupts **disabled** when calling switch

```
lock() {  
    disable interrupts  
    while (status != FREE) {  
        add thread to queue of  
            threads waiting for lock  
        enable interrupts  
        switch to next ready thread  
        disable interrupts  
    }  
    status = BUSY  
    enable interrupts  
}
```

```
unlock() {  
    disable interrupts  
    status = FREE  
    if (any thread is waiting  
        for this lock) {  
        move waiting thread to  
            ready queue  
    }  
    enable interrupts  
}
```

- Switch without re-enabling interrupts ?!

lock	disable interrupts
do stuff	do stuff
unlock	enable interrupts

Leave interrupts **disabled** when calling switch

```
lock() {  
    disable interrupts  
    while (status != FREE) {  
        add thread to queue of  
            threads waiting for lock  
        enable interrupts  
        switch to next ready thread  
         disable interrupts  
    }  
    status = BUSY  
    enable interrupts  
}
```

```
unlock() {  
    disable interrupts  
    status = FREE  
    if (any thread is waiting  
        for this lock) {  
        move waiting thread to  
            ready queue  
    }  
    enable interrupts  
}
```

- What can lock() assume about the state of interrupts after switch returns?
- How does lock() wake up from switch?

Switch invariant (for uniprocessors)

- All threads promise to have interrupts disabled when calling switch
- All threads can assume that interrupts are “still” disabled when switch returns

Thread A

```
enable interrupts  
}
```

<user code runs>

```
lock() {  
    disable interrupts  
    ...  
    swapcontext
```

```
}
```

```
unlock() (move thread A to ready  
queue)
```

```
yield() {  
    disable interrupts
```

```
swapcontext  
back from swapcontext  
enable interrupts
```

```
}
```

Thread B (currently holds lock)

```
yield() {  
    disable interrupts  
    swapcontext
```

```
back from swapcontext  
enable interrupts
```

```
}
```

```
unlock() (move thread A to ready  
queue)
```

```
yield() {  
    disable interrupts
```

```
swapcontext
```

Lock implementation #2

(uniprocessor, no busy waiting)

```
lock() {  
    disable interrupts  
    while (status != FREE) {  
        add thread to queue of  
            threads waiting for lock  
        enable interrupts  
        switch to next ready thread  
        disable interrupts  
    }  
    status = BUSY  
    enable interrupts  
}
```

```
unlock() {  
    disable interrupts  
    status = FREE  
    if (any thread is waiting  
        for this lock) {  
        move waiting thread to  
            ready queue  
    }  
    enable interrupts  
}
```

Lock implementation #3 (uniprocessor, no busy waiting, handoff)

```
lock() {  
    disable interrupts  
if while (status != FREE) {  
        add thread to queue of  
            threads waiting for lock  
  
        switch to next ready thread  
    } else {  
        status = BUSY  
    }  
    enable interrupts  
}
```

```
unlock() {  
    disable interrupts  
    status = FREE  
    if (any thread is waiting  
        for this lock) {  
        move waiting thread to  
            ready queue  
        status = BUSY  
    }  
    enable interrupts  
}
```